National University of Computer and Emerging Sciences

Karachi Campus

Data Science for Software Engineering (SE-4096)
Semester: Spring-2026
Date: Thursday, 19th Feb, 2026
Course Instructor(s) Dr Muhammad Yaseen Khan

Quiz 01
Total Time: 15 minutes
Total Points: 4
Total Questions: 02

Student Name	Roll Number	Section	Signature
_____	_____	_____	_____

Read the following scenario carefully and answer the questions below.

You are the Lead Software Test Engineer in a software company developing web and mobile applications. Recently, releases have been delayed due to missed defects, unstable builds, and uneven tester productivity. Some testers consistently execute fewer test cases and report fewer defects than others, affecting overall software quality.

Testing activities and results are recorded in repositories hosted on GitHub, along with issue trackers and CI/CD logs that store defect reports, build outcomes, execution times, and contributor information. You decide to apply Data Science for Software Engineering practices to analyze this data and generate productivity reports to support decisions such as workload redistribution, training, or probation for consistently low-performing testers.

— o —

There are potentially many solutions, some of which are itemized below (in a limited time, students may not write all of them; however, for comprehension, most of them are maintained here).

CLO 1: Students should be able to articulate a comprehensive understanding of Data Science, encompassing its fundamental concepts, methodologies, and practical applications in the Software Engineering discipline.

Question 1. Explain why applying Data Science for Software Engineering is necessary in this scenario and why testing-related repository data is important for evaluating tester productivity and improving software quality. [2 marks]

Using DS4SE (Data Science for Software Engineering) transforms raw data into actionable information to eliminate bottlenecks in your release cycle. Here's why this approach and repository data are important in this scenario:

- **Objective Performance Benchmarking:** Data science enables you to focus on specific, data-driven metrics rather than "guessing." By applying clustering or regression to GitHub and CI/CD data, you can understand the complexity of the work, so that testers working on difficult modules aren't mistaken for testers working on easier tasks.
  Hint: Clustering on the data will bring the projects (with easier/difficult tasks) in logically and semantically equivalent groups (and may be so with other latent (hidden) features). Similarly, productivity measurement is a quantitative task (say you will assign some score out of 100 to every tester). Predicting that score is a regression task.

- **Identifying the Root Cause:** Simply assuming the tester is "slow" isn't enough. DS4SE techniques allow you to combine execution time and build stability logs. This reveals that it's likely not the tester's error, but rather an unstable system or "flaky" infrastructure that's holding you back.

- **Focus on Quality Over Quantity:** By checking the quality of defect leakage (bugs that remain in testing and make it into production) on GitHub and using NLP (Natural Language Processing), you can determine true "software reliability" rather than just counting bugs. That is the narration of bug will be different during testing and production phases. NLP will help to distinguish between older and newer types of software using the bugs' narrations.

- **Predictive Release Management:** By analyzing historical CI/CD logs, you can build models that predict whether a release will be delayed. This allows you to avoid deadlines by training or shifting resources ahead of time.

- **Basing decisions based on evidence:** When you place someone on probation or provide training, repository data serves as a transparent audit trail. This shows where performance issues are, allowing management decisions to be based on solid evidence.

- **Explanation of Build Health:** Testing data is a diagnostic tool for the entire development lifecycle. Modules that repeatedly fail in the repository logs reveal where software quality is really declining, so you can locate/focus more on where it matters.

CLO 2: Students should be able to design and implement data pre-processing pipelines to clean, transform, and prepare raw data for analysis, ensuring data quality and integrity.

Question 2. Explain how you would collect, preprocess, and analyze the testing data to compute productivity metrics and identify underperforming testers. [2 marks]

In this scenario, using the DS4SE pipeline is essential to clean the raw data and produce accurate results. The following steps ensure data quality and integrity:

- **Data Collection (GitHub & CI/CD Integration):** First, raw data (commit history, bug reports, execution time, and build results) is collected from the GitHub API and CI/CD logs. The purpose is to obtain a complete record of each tester's activity.

- **Data Cleaning** (Noise Removal): Raw logs often contain "flaky tests" (those that fail due to system errors) or duplicate bug reports. The cleaning step removes these redundant records to avoid skewing productivity metrics. This also involves file names changes, deleted branches, in-complete logs etc.

- **Data Transformation & Normalization**: The complexity of every test case is not the same. Transformation adds a "Test Complexity Factor" so that the score of the tester performing the most difficult task is normalized according to their effort.

- **Handling Missing Values:** If a tester's execution time is not logged, data gaps are filled using "Imputation" techniques (e.g., average or median values) so that the analysis is not incomplete.

- **Feature Engineering:** New metrics are created from the raw data, such as "Defect Rejection Rate" (how many bugs were rejected) and "Bug Severity Score," which reflect the quality of the work rather than just a count. Similarly, date/timestamp can be used for creating pivots for quarterly/weekly analysis.

- **Statistical Analysis & Outlier Detection:** By applying statistical models (like Z-score or Clustering) on pre-processed data, testers are identified whose performance is significantly lower than the team average.

- **Integrity Check (Cross-validation):** Before final analysis, the data is cross-verified to ensure that the observed "low productivity" was not due to any technical issue, so that the decision is always based on solid and healthy data.